17 Monitoring Spring with JMX

This chapter covers

- Working with Actuator endpoint MBeans
- Exposing Spring beans as MBeans
- Publishing notifications

For over a decade and a half, Java Management Extensions (JMX) has been the standard means of monitoring and managing Java applications. By exposing managed components known as MBeans (managed beans), an external JMX client can manage an application by invoking operations, inspecting properties, and monitoring events from MBeans.

We'll start exploring Spring and JMX by looking at how Actuator endpoints are exposed as MBeans.

17.1 Working with Actuator MBeans

By default, all Actuator endpoints are exposed as MBeans. But, starting with Spring Boot 2.2, JMX itself is disabled by default. To enable JMX in your Spring Boot application, you can set `spring.jmx.enabled` to `true`. In application.yml, this would look like this:

```
spring:
  jmx:
    enabled: true
```

With that property set, Spring support for JMX is enabled. And with it, the Actuator endpoints are all exposed as MBeans. You can use any JMX client you wish to connect with Actuator endpoint MBeans. Using JConsole, which comes with the Java Development Kit, you'll find Actuator MBeans listed under the `org.springframework.boot` domain, as shown in figure 17.1.

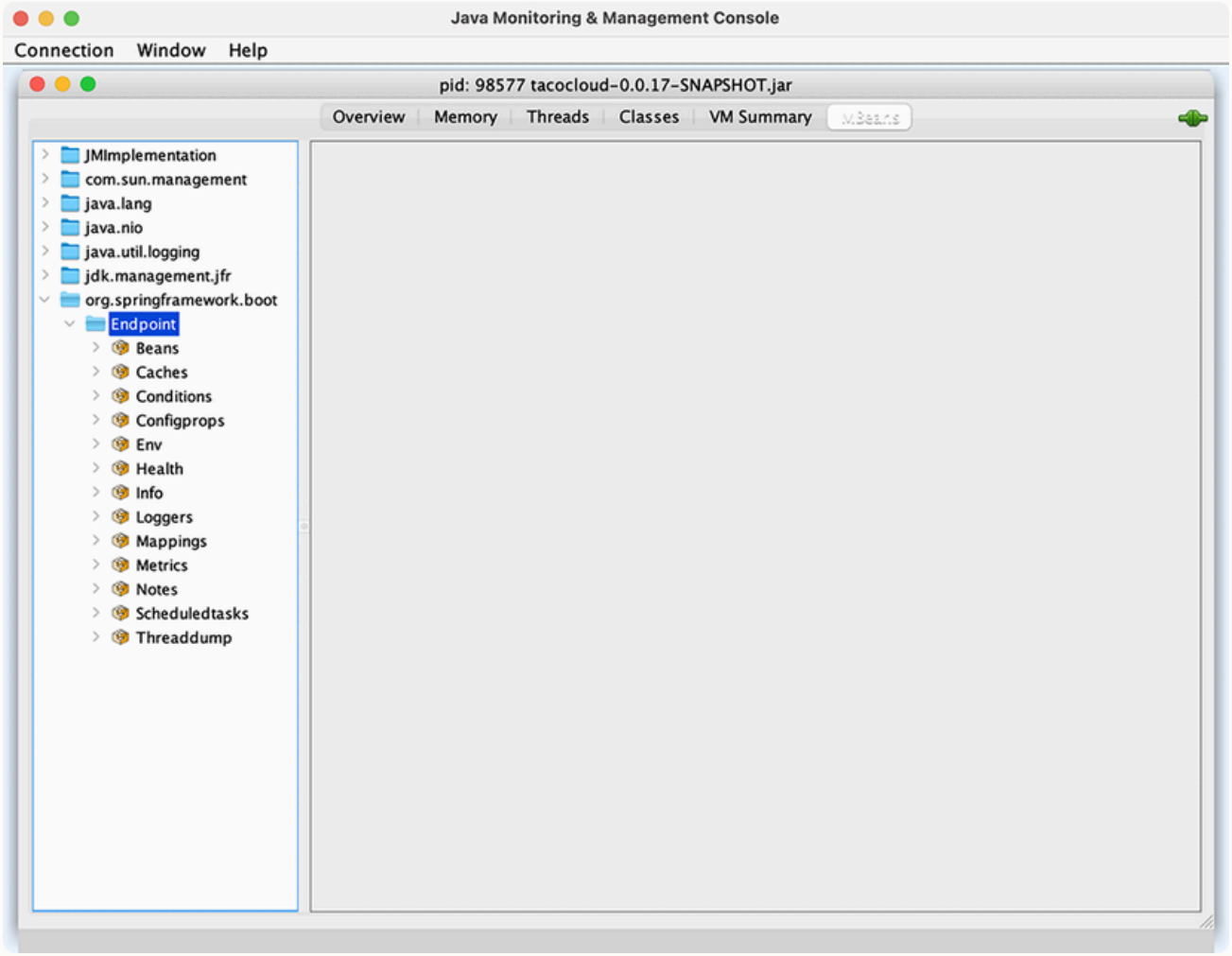


Figure 17.1 Actuator endpoints are automatically exposed as JMX MBeans.

One thing that's nice about Actuator MBean endpoints is that they're all exposed by default. There's no need to explicitly include any of them, as you had to do with HTTP. You can, however, choose to narrow down the choices by setting `management.endpoints.jmx.exposure.include` and `management.endpoints.jmx.exposure.exclude`. For example, to limit Actuator endpoint MBeans to only the /health, /info, /bean, and /conditions endpoints, set `management.endpoints.jmx.exposure.include` like this:

```
management:
  endpoints:
    jmx:
      exposure:
        include: health,info,bean,conditions
```

Or, if there are only a few you want to exclude, you can set `management.endpoints.jmx.exposure.exclude` like this:

```
management:
  endpoints:
    jmx:
```

```
    exposure:
      exclude: env,metrics
```

Here, you use `management.endpoints.jmx.exposure.exclude` to exclude the /env and /metrics endpoints. All other Actuator endpoints will still be exposed as MBeans.

To invoke the managed operations on one of the Actuator MBeans in JConsole, expand the endpoint MBean in the left-hand tree, and then select the desired operation under Operations.

For example, if you'd like to inspect the logging levels for the `tacos.ingredients` package, expand the `Loggers` MBean and click on the operation named `loggerLevels`, as shown in figure 17.2. In the form at the top right, fill in the Name field with the package name (`org.springframework.web`, for example), and then click the `loggerLevels` button.

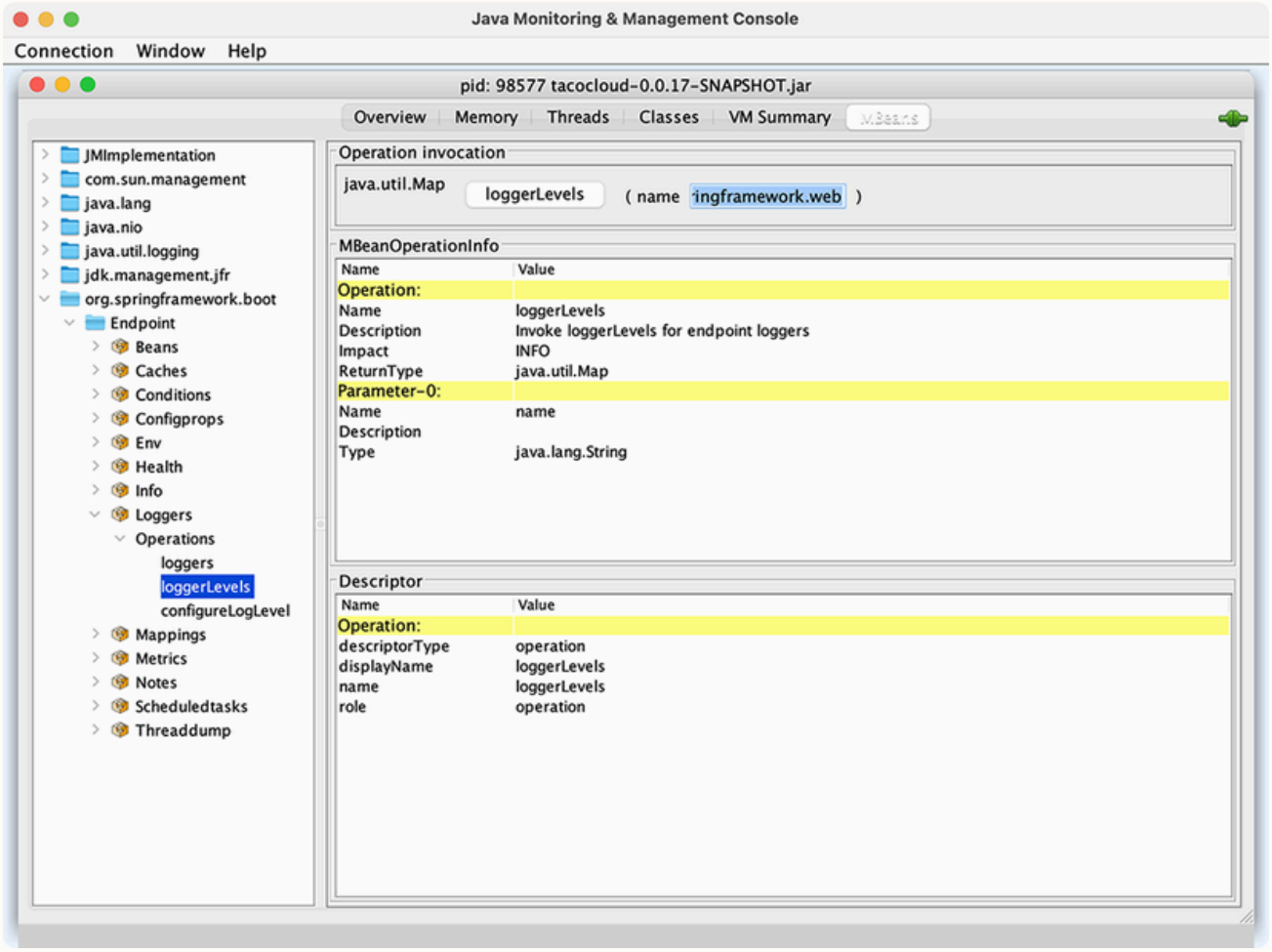


Figure 17.2 Using JConsole to display logging levels from a Spring Boot application

After you click the `loggerLevels` button, a dialog box will pop up, showing you the response from the /loggers endpoint MBean. It might look a little like figure 17.3.

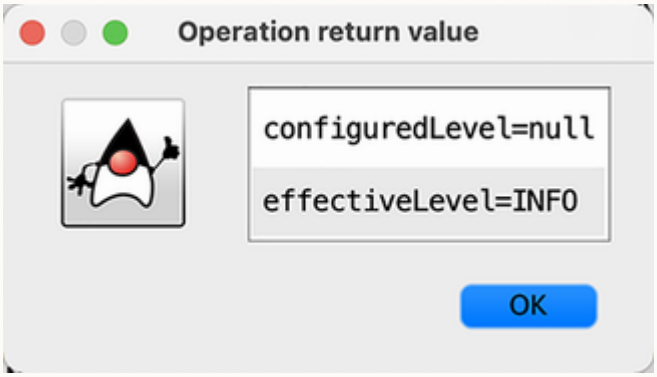


Figure 17.3 Logging levels from the /loggers endpoint MBean displayed in JConsole

Although the JConsole UI is a bit clumsy to work with, you should be able to get the hang of it and use it to explore any Actuator endpoint in much the same way. If you don't like JConsole, that's fine—there are plenty of other JMX clients to choose from.

17.2 Creating your own MBeans

Spring makes it easy to expose any bean you want as a JMX MBean. All you must do is annotate the bean class with `@ManagedResource` and then annotate any methods or properties with `@ManagedOperation` or `@ManagedAttribute`. Spring will take care of the rest.

For example, suppose you want to provide an MBean that tracks how many tacos have been ordered through Taco Cloud. You can define a service bean that keeps a running count of how many tacos have been created. The following listing shows what such a service might look like.

Listing 17.1 An MBean that counts how many tacos have been created

```
package tacos.jmx;

import java.util.concurrent.atomic.AtomicLong;
import org.springframework.data.rest.core.event.AbstractRepositoryEventListener;
import org.springframework.jmx.export.annotation.ManagedAttribute;
import org.springframework.jmx.export.annotation.ManagedOperation;
import org.springframework.jmx.export.annotation.ManagedResource;
import org.springframework.stereotype.Service;
import tacos.Taco;
import tacos.data.TacoRepository;

@Service
@ManagedResource
public class TacoCounter
```

```
        extends AbstractRepositoryEventListener<Taco> {

    private AtomicLong counter;
    public TacoCounter(TacoRepository tacoRepo) {
      tacoRepo
          .count()
          .subscribe(initialCount -> {
             this.counter = new AtomicLong(initialCount);
          });
    }

    @Override
    protected void onAfterCreate(Taco entity) {
      counter.incrementAndGet();
    }

    @ManagedAttribute
    public long getTacoCount() {
      return counter.get();
    }

    @ManagedOperation
    public long increment(long delta) {
      return counter.addAndGet(delta);
    }

  }
```

The `TacoCounter` class is annotated with `@Service` so that it will be picked up by component scanning and an instance will be registered as a bean in the Spring application context. But it's also annotated with `@ManagedResource` to indicate that this bean should also be an MBean. As an MBean, it will expose one attribute and one operation. The `getTacoCount()` method is annotated with `@ManagedAttribute` so that it will be exposed as an MBean attribute, whereas the `increment()` method is annotated with `@ManagedOperation`, exposing it as an MBean operation. Figure 17.4 shows how the `TacoCounter` MBean appears in JConsole.

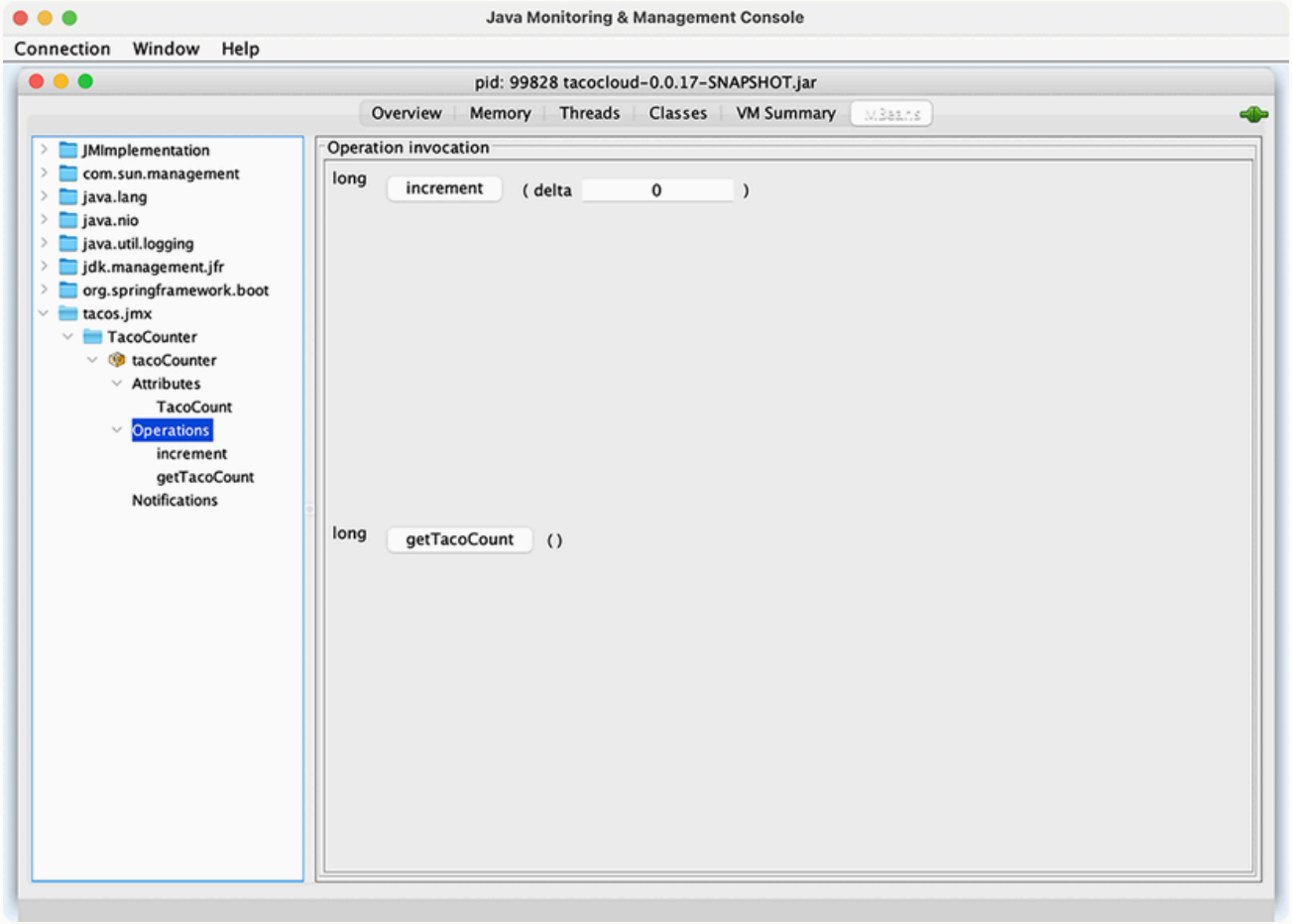


Figure 17.4 `TacoCounter`'s operations and attributes as seen in JConsole

`TacoCounter` has another trick up its sleeve, although it has nothing to do with JMX. Because it extends `AbstractRepositoryEventListener`, it will be notified of any persistence events when a `Taco` is saved through `TacoRepository`. In this particular case, the `onAfterCreate()` method will be invoked anytime a new `Taco` object is created and saved, and it will increment the counter by one. But `AbstractRepositoryEventListener` also offers several methods for handling events both before and after objects are created, saved, or deleted.

Working with MBean operations and attributes is largely a pull operation. That is, even if the value of an MBean attribute changes, you won't know until you view the attribute through a JMX client. Let's turn the tables and see how you can push notifications from an MBean to a JMX client.

17.3 Sending notifications

MBeans can push notifications to interested JMX clients with Spring's `NotificationPublisher`. `NotificationPublisher` has a single `sendNotification()` method that, when given a `Notification` object, publishes the notification to any JMX clients that have subscribed to the MBean.

For an MBean to be able to publish notifications, it must implement the `NotificationPublisherAware` interface, which requires that a `setNotificationPublisher()` method be implemented. For example, suppose you want to publish a notification for every 100 tacos that are created. You can change the `TacoCounter` class so that it implements `NotificationPublisherAware` and uses the injected `NotificationPublisher` to send notifications for every 100 tacos that are created. The following listing shows the changes that must be made to `TacoCounter` to enable such notifications.

Listing 17.2 Sending notifications for every 100 tacos

```
package tacos.jmx;

import java.util.concurrent.atomic.AtomicLong;
import org.springframework.data.rest.core.event.AbstractRepositoryEventListener;
import org.springframework.jmx.export.annotation.ManagedAttribute;
import org.springframework.jmx.export.annotation.ManagedOperation;
import org.springframework.jmx.export.annotation.ManagedResource;
import org.springframework.stereotype.Service;

import org.springframework.jmx.export.notification.NotificationPublisher;
import org.springframework.jmx.export.notification.NotificationPublisherAware;
import javax.management.Notification;

import tacos.Taco;
import tacos.data.TacoRepository;

@Service
@ManagedResource
public class TacoCounter
      extends AbstractRepositoryEventListener<Taco>
      implements NotificationPublisherAware {


  private AtomicLong counter;
  private NotificationPublisher np;

  @Override
  public void setNotificationPublisher(NotificationPublisher np) {
    this.np = np;
  }

  ...

  @ManagedOperation
```

```
  public long increment(long delta) {
    long before = counter.get();
    long after = counter.addAndGet(delta);
    if ((after / 100) > (before / 100)) {
      Notification notification = new Notification(
          "taco.count", this,
          before, after + "th taco created!");
      np.sendNotification(notification);
    }
    return after;
  }

}
```

In the JMX client, you'll need to subscribe to the `TacoCounter` MBean to receive notifications. Then, as tacos are created, the client will receive notifications for each century count. Figure 17.5 shows how the notifications may appear in JConsole.

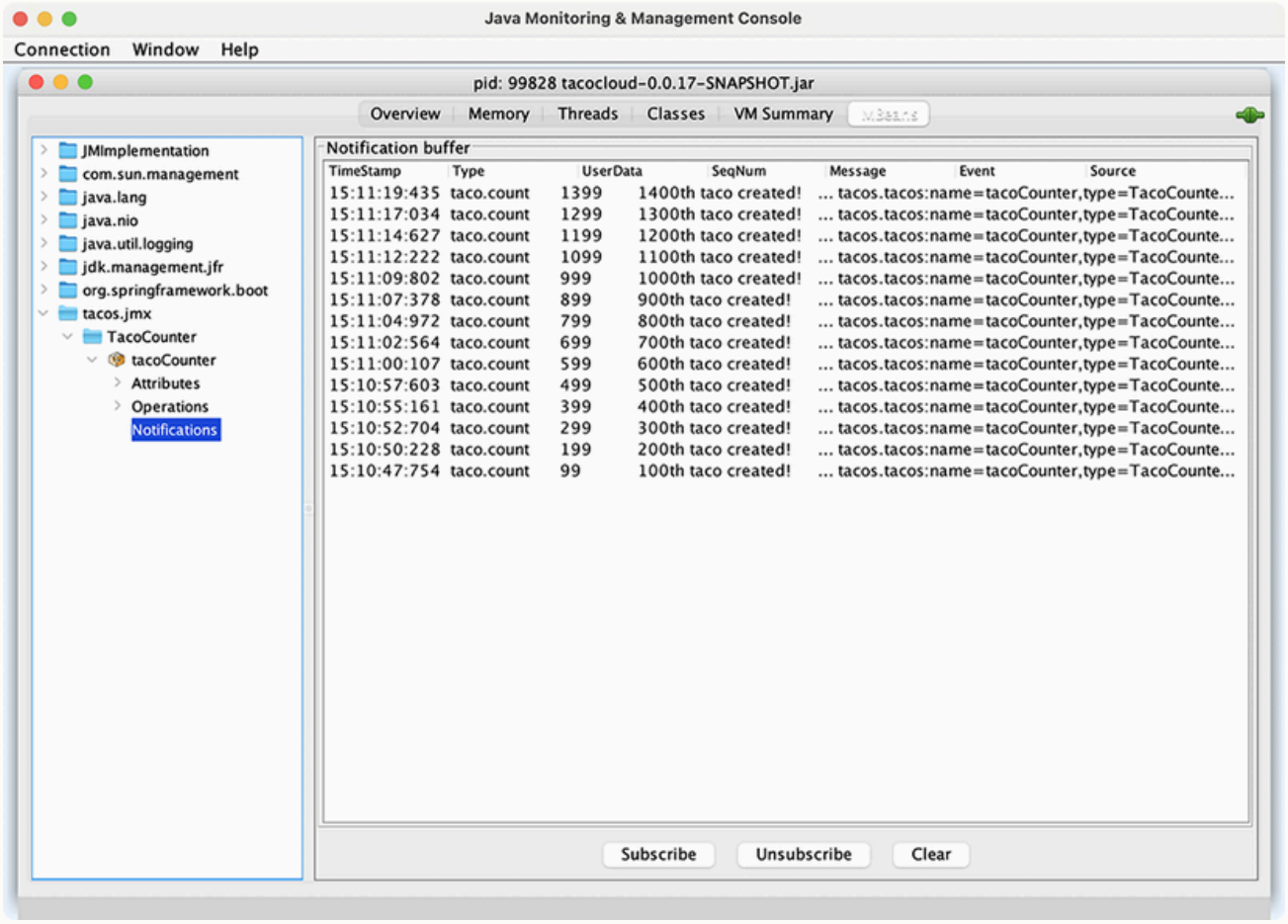


Figure 17.5 JConsole, subscribed to the `TacoCounter` MBean, receives a notification for every 100 tacos that are created.

Notifications are a great way for an application to actively send data and alerts to a monitoring client without requiring the client to poll managed attributes or invoke managed operations.

Summary

- Most Actuator endpoints are available as MBeans that can be consumed using any JMX client.
- Spring automatically enables JMX for monitoring beans in the Spring application context.
- Spring beans can be exposed as MBeans by annotating them with `@ManagedResource`. Their methods and properties can be exposed as managed operations and attributes by annotating the bean class with `@ManagedOperation` and `@ManagedAttribute`.
- Spring beans can publish notifications to JMX clients using `NotificationPublisher`.

Previous chapter
< 16 Administering Spring

Next chapter
18 Deploying Spring >